Algorithm Analysis

Liting Zhang

University of West Florida

Outline

- Motivation: why analyze algorithms
- Two approaches: empirical vs theoretical
- Computational complexity: resources and growth with problem size
- Time complexity: best, worst, average cases
- Space complexity
- Bounds and asymptotic notation: O, Ω, Θ
- Examples: linear search, binary search, selection sort
- Practices and exercises: finding $T(n)$ and choosing bounds
- Amortized analysis: when single-operation cases are not enough

Learning goals

After this lecture, you should be able to

- Explain empirical vs theoretical analysis and when to use each
- Define problem size and a cost model for counting operations
- Analyze best, worst, and average cases for simple loops
- Explain time vs space complexity
- Use O, Ω, Θ correctly and avoid mixing bounds with cases
- Derive complexity for linear search, binary search, and selection sort
- Explain amortized analysis using dynamic array resizing as a motivating example

Motivation

Why analyze algorithms?

- To compare algorithms and select the best algorithm for a given problem
- To predict how runtime grows when input size grows
- To choose data structures and implementations that scale
- To explain performance independently of hardware and programming language

Types of Analysis

Empirical analysis

Empirical analysis means you implement, run, and measure performance.

- Pros: easy to start, gives real measurements
- Cons: hardware dependent, compiler flags matter, input distributions matter
- Good for: validating expectations, comparing real implementations

Empirical timing: C++ chrono example

```cpp
#include <chrono>
#include <iostream>
#include <vector>

int main() {
  std::vector<int> v(1000000, 1);

  auto start = std::chrono::high_resolution_clock::now();

  long long sum = 0;
  for (int x : v) sum += x;

  auto stop = std::chrono::high_resolution_clock::now();
  auto ms =
  ↪  std::chrono::duration_cast<std::chrono::milliseconds>(stop
  ↪  - start);

  std::cout << "sum=" << sum << " time_ms=" << ms.count() <<
  ↪  "\n";
}
```

Theoretical analysis

Theoretical analysis uses math to estimate required resources.

- Hardware and software independent
- Focuses on growth as problem size increases
- Produces complexity functions like $T(n)$ and asymptotic bounds

This is what we mostly do in data structures and algorithms.

Computational Complexity

Definition

The computational complexity of an algorithm estimates required resources as the problem size increases.

Examples of resources:

- time
- space (memory)
- power consumption
- network traffic

Problem size

Before analyzing, define the problem size n.

- array algorithms: n = number of elements
- string algorithms: n = length of the string
- graphs: n might involve vertices V and edges E

A good analysis always states what n means.

Cost model

We count a basic set of operations.

- comparisons
- assignments
- arithmetic on fixed-size integers
- array index access

Then we express total work as a function $T(n)$.

Time and Space Complexity

Time complexity cases

For the same algorithm, runtime can vary with input pattern.

- Best case: minimum resources for any input of size n
- Worst case: maximum resources for any input of size n
- Average case: expected resources under an input distribution

Space complexity

Space complexity measures memory usage as a function of n.

- Input storage is often not counted (depends on convention)
- Auxiliary space counts additional memory used by the algorithm
- Recursion adds implicit stack space

Constant time operations

An operation is $O(1)$ if it runs in the same time regardless of n.

- assignments: x = 10
- arithmetic on fixed-size integers: z = a + b
- comparisons: if (a ¿ b)
- array access by index: x = arr[i]

Not $O(1)$ examples:

- loops that depend on n
- string concatenation that copies characters
- recursion depth dependent on input size

Bounds and Asymptotic Notation

Upper and lower bounds

Bounds describe growth rates, not a specific input scenario.

- Lower bound: a function that is always at most the runtime
- Upper bound: a function that is always at least the runtime

We prefer tight bounds to compare algorithms fairly.

Bounds vs cases

Important: bounds and cases are not the same.

- Best, worst, average are about input scenarios
- O, Ω, Θ are about mathematical bounding of a function
- Do not treat an upper bound as the worst case automatically
- Do not treat a lower bound as the best case automatically

Cases and bounds table

Concept	What it describes	Example wording
Best case	Most favorable input of size n	runtime is smallest when key is first
Worst case	Most unfavorable input of size n	runtime is largest when key is absent
Average case	Expected runtime under a distribution	expected comparisons over random position
O bound	upper bound on growth of a function	$T(n)$ grows no faster than n^2
Ω bound	lower bound on growth of a function	$T(n)$ grows at least as fast as n
Θ bound	tight bound on growth of a function	$T(n)$ grows like $n \log n$

Asymptotic notation intuition

Why we use asymptotic notation:

- Exact formulas can be complicated and non-intuitive
- We want a simple description of growth for large n
- We ignore constants and lower-order terms to focus on scaling

Visualization idea for O, Ω, Θ

$O(g(n))$ means $T(n)$ is at or below a constant multiple of $g(n)$ for large n

Examples

Linear search efficiency

Linear search checks elements from the beginning until: key is found or end is reached.
Runtime:

- best case: $O(1)$ (key is first element)
- worst case: $O(n)$ (key not found or last element)
- average case: $O(n)$

Linear search pseudocode

```
1 LinearSearch(numbers, numbersSize, key) {
2   for (i = 0; i < numbersSize; ++i) {
3     if (numbers[i] == key) {
4       return i
5     }
6   }
7   return -1
8 }
```

Binary search efficiency

Binary search works on sorted arrays: repeatedly check the middle element and halve the search range.
Runtime:

- best case: $O(1)$ (key is at middle)
- worst case: $O(\log n)$
- average case: $O(\log n)$

Binary search pseudocode

```
1  BinarySearch(numbers, numbersSize, key) {
2    low = 0
3    high = numbersSize - 1
4    while (high >= low) {
5      mid = (high + low) / 2
6      if (numbers[mid] < key) {
7        low = mid + 1
8      }
9      else if (numbers[mid] > key) {
10       high = mid - 1
11     }
12     else {
13       return mid
14     }
15   }
16   return -1
17 }
```

Selection sort: what to expect

Selection sort repeatedly selects the minimum and swaps it into position.
Runtime:

- comparisons: about $n(n-1)/2$
- overall: $O(n^2)$ time
- space: $O(1)$ auxiliary space

Selection sort (C++)

```cpp
void selectionSort(int* arr, int size) {
  for (int i = 0; i < size - 1; ++i) {
    int min = i;
    for (int j = i + 1; j < size; ++j) {
      if (arr[j] < arr[min]) {
        min = j;
      }
    }
    if (min != i) {
      std::swap(arr[i], arr[min]);
    }
  }
}
```

Practice 1: analyze search

Find $T(n)$ and give the worst-case bound.

```
1 int search(int* arr, int size, int val) {
2   for (int i = 0; i < size; ++i) {
3     if (arr[i] == val) return i;
4   }
5   return -1;
6 }
```

Practice 1 Answer

Let n be size.

- Worst case: compare val to every element, so about n comparisons
- $T(n)$ is linear in n
- Worst-case bound: $O(n)$
- Best case: first element matches, so $O(1)$

Practice 2: selection sort bound

Give the dominant term for comparisons and the final time complexity.

```
1  void selectionSort(int* arr, int size) {
2    for (int i = 0; i < size - 1; ++i) {
3      int min = i;
4      for (int j = i + 1; j < size; ++j)
5        if (arr[j] < arr[min])
6          min = j;
7      if (min != i)
8        std::swap(arr[i], arr[min]);
9    }
10 }
```

Practice 2 Answer

- Inner loop does about (size - i - 1) comparisons
- Total comparisons: (n-1) + (n-2) + . . . + 1 = n(n-1)/2
- Dominant term: $n^2/2$
- Time complexity: $O(n^2)$

Practice 3: isReverse

```
1 bool isReverse(const char* arr1, const char* arr2, int length)
  ↪  {
2   for (int i = 0; i < length; ++i) {
3     if (arr1[i] != arr2[length - 1 - i]) {
4       return false;
5     }
6   }
7   return true;
8 }
```

- What is the problem size
- What is the worst-case time bound, and which notation would you use

Practice 3 Answer

- Problem size: length
- Worst case: arrays are reverse of each other, so loop runs length times
- Time complexity: $O(n)$ where n is length
- Also: best case can be $O(1)$ if mismatch happens immediately

Common mistake: bounds vs worst-case

For isReverse:

- worst-case runtime function grows linearly
- then we apply an upper bound notation: $O(n)$

Do not say:

- upper bound equals worst case

Correct idea:

- worst-case is a function; O is a bound on that function

Amortized Analysis

Amortized analysis motivation

Amortized analysis studies the average cost of a sequence of operations.

- Used when best, worst, and average per operation do not describe performance well
- A few expensive operations may be rare but unavoidable
- The total cost over many operations is still manageable

Where amortized analysis appears

Examples:

- dynamic arrays (resizing)
- hash tables (rehashing)
- self-balancing BSTs (rebalancing steps)
- splay trees (splaying)

Dynamic array resizing story

When a dynamic array is full:

- allocate a bigger array, often double capacity
- copy all elements
- delete the old array

A single resize is expensive, but it does not happen every push. Over many pushes, average cost per push is constant amortized time.

Amortized methods

Three common methods:

- Aggregate analysis
- Accounting method
- Potential method

In this course, the key goal is recognizing when amortized reasoning is needed.

Review

Summary

- Empirical analysis measures real code, but depends on environment
- Theoretical analysis models growth with problem size
- Time complexity depends on best, worst, and average input scenarios
- Asymptotic notation describes bounds on growth: O, Ω, Θ
- Linear search is $O(n)$ worst case; binary search is $O(\log n)$ worst case
- Selection sort is $O(n^2)$ time with $O(1)$ auxiliary space
- Amortized analysis explains sequences of operations, such as dynamic array resizing

Review questions

- What is the difference between empirical and theoretical analysis
- What is problem size for an array algorithm
- Why do we ignore constants in asymptotic notation
- For binary search, why is the runtime logarithmic
- Why is selection sort quadratic
- What does amortized mean in the context of dynamic arrays